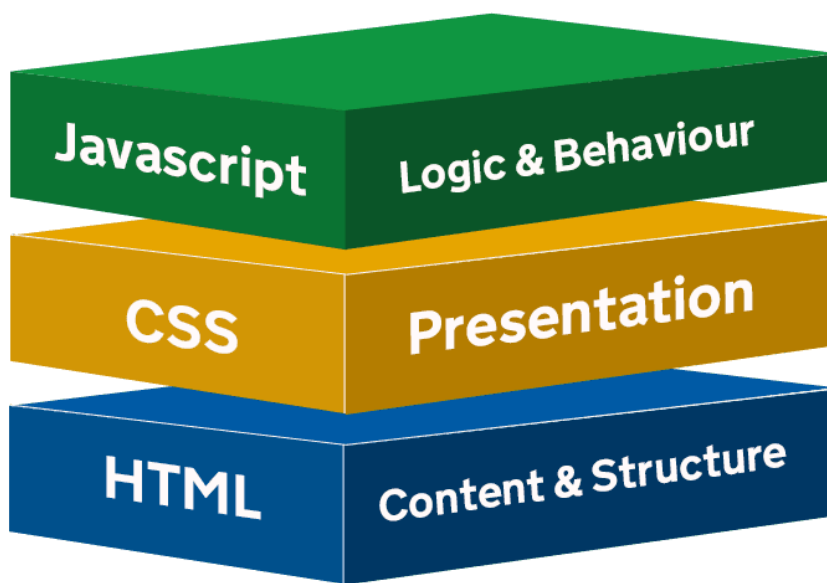


TW11 Beyond HTML 1

Links:

- <https://codepen.io/calebdwilliams/pen/rRNJPJ>
- <https://html.spec.whatwg.org/multipage/custom-elements.html>
- <https://css-tricks.com/an-introduction-to-web-components/>
- <https://github.com/mdn/web-components-examples>



[list2tab]

- SGML Standardized Generalized Markup Lang
 - base form **meta**-language to define other markup languages
 - **Declarative** specify structure & attributes ✗ how to process
 - **Rigorous** strict def of structure **way tag**
open/close/nested, what kind permitted in diff places
 - DTD **Document Type Definition** → diff families within this umbrella, each /w own **custom tags** interpretation & document behaviour as per needs
 - ✗ strict tags **HTML2 aligns with SGML, HTML4**
✓ compliant, HTML5 ✗ but designed to be forgiving of errors for

accessibility

- XML eXtensible Markup lang

- Human-readable: simple general **✗** predefined usable **custom tags**
→ multiple applications defined

- Machine readable: **✓** complex data structures & relationships

MathML

RSS feeds are primarily used to get information about latest updates on a given page(News, blogs)

Atom, SVG Scalable Vector Graphics

- **XHTML = HTML4 + XML Namespaces** allow inter-operability, modular, extensible

- HTML5

- multimedia support, canvases
- Enhanced APIs for better browser interaction.
- human-machine readable, backward compatible (older tags)
- **✗ XML/SGML** **✓ own parsing rules**
- maintained by **WHATWG** Web Hypertext Application Technology Working Group **W3C**
- Standard → want New tags/features in ur site? → **JavaScript**, Custom elements **API supported by browsers** → Semantics may **✗** be well thought for **search engine**

```
<!ELEMENT book (title, author, chapter)> // stricter  
than HTML
```

XML Example - RSS feeds

```
<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">
<channel>
  <title>RSS Title</title>
  <description>This is an example of an RSS
feed</description>
  <link>http://www.example.com/main.htm</link>
  <copyright>2020 Example.com All rights
reserved</copyright>
  <lastBuildDate>Mon, 06 Sep 2010 00:01:00
+0000</lastBuildDate>
  <pubDate>Sun, 06 Sep 2009 16:20:00 +0000</pubDate>
  <ttl>1800</ttl>

  <item>
    <title>Example entry</title>
    <description>Here is some text containing an
interesting description</description>
    <link>http://www.example.com/blog/post/</link>
    <guid isPermaLink="false">7bd204c6-1655-4c27-
aeee-53f933c5395</guid>
    <pubDate>Sun, 06 Sep 2009 16:20:00
+0000</pubDate>
  </item>

</channel>
</rss>
```

```
<svg version="1.1"
width="300" height="200"
xmlns="http://www.w3.org/2000/svg">
  <rect width="100%" height="100%" fill="red" />
  <circle cx="150" cy="100" r="80" fill="green" />
  <text x="150" y="125" font-size="60" // (150, 225)
    text-anchor="middle" fill="white">SVG</text>
</svg>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE note {
<!ELEMENT title (#PCDATA)> // stricter than HTML
<!ELEMENT body (#PCDATA)>
<!ELEMENT item (title, description, link, guid,
pubDate)>
}>

<body>this is body</body>
```

JavaScript

[list2node]

- High lvl prog lang
 - object orientation **class**
 - dynamic typing **variables can hold different types** **Xinit**
type declaration
 - function composition
 - imperative procedures
 - Web browser dedicated **js** engine
 - Event-driven Handles **asynchronous** events efficiently (user clicks)
- APIs
 - **text, dates, RegEx**
 - dict/list/array DS

- DOM **document object model** real-time interaction /w webpage browser environment
- ❌ native IO or file access but through external APIs

JS

```
const para = document.querySelector('p');
para.addEventListener('click', updateName);

function updateName() {
  let name = prompt('Enter a new name');
  para.textContent = 'Player 1: ' + name;
}
```

HTML

```
<button id="button_A">Press me</button>
<h3 id="heading_A"></h3>
```

JS

```
const buttonA = document.querySelector('#button_A');
const headingA =
document.querySelector('#heading_A');

buttonA.onclick = function() {
  let name = prompt('What is your name?');
  alert('Hello ' + name + ', nice to see you!');
  headingA.textContent = 'Welcome ' + name;
}
```

Web components

- ❌ HTML5 ✅ XML allows arbitrary namespaces & tag definitions
 - **Render** display details how shown
 - **States** checkbox - blank/checked (custom?)
 - Customized built-in/Autonomous custom element
 - Limited Standardization ✅ reuse
 - Modular design

[list2tab]

- shadow DOM API

keeps styling of component separate from rest of page (restrict scope of modification of content)

```
JS
const shadowRoot =
document.getElementById('example').attachShadow({
mode: 'open' });
shadowRoot.innerHTML = `<style>
button {
color: tomato;
}
</style>
<button id="button">This will use the CSS color
tomato <slot></slot></button>`;
```

- Custom Elements (JS API)

Enhanced APIs for better browser interaction.

```
<flag-icon country="nl"></flag-icon>
// declare a class for the custom element,
extending HTMLElement:

class FlagIcon extends HTMLElement {
  constructor() {
    super();
    this._countryCode = null;
    this._img = document.createElement('img'); //
create an img element
    this.attachShadow({ mode: 'open'
}).appendChild(this._img); // use shadow DOM to
encapsulate styles and markup
  }

  static get observedAttributes() {
    return ["country"];
  }

  attributeChangedCallback(name, oldValue,
newValue) {
    // name will always be "country" due to
observedAttributes
    this._countryCode = newValue;
    this._updateRendering();
  }

  connectedCallback() {
    this._updateRendering();
  }

  get country() {
    return this._countryCode;
  }
}
```

```

    set country(v) {
        this.setAttribute("country", v);
    }

    _updateRendering() {
        // Check if element is in a document with
        browsing context
        if (!this.ownerDocument.defaultView) return;
        if (this._countryCode) {
            // Use a source path for flags, here using a
            free public CDN for country flags as example
            this._img.src =
`https://flagcdn.com/w80/${this._countryCode.toLower
erCase()}.png`;
            this._img.alt = `Flag of
${this._countryCode.toUpperCase()}`;
            this._img.style.width = '80px';
            this._img.style.height = 'auto';
            this._img.style.display = 'block';
        } else {
            this._img.removeAttribute('src');
            this._img.alt = '';
        }
    }
}

```

```

customElements.define("flag-icon", FlagIcon);
// The parser, whenever it sees the flag-icon tag,
will construct a new instance of our FlagIcon
class, and tell our code about its new country
attribute, which we then use to set the element's
internal state and update its rendering (when
appropriate).
const flagIcon = document.createElement("flag-
icon")
flagIcon.country = "jp"

```



```
document.body.appendChild(flagIcon)
// use the custom element constructor itself.
above code is equivalent to:
```

```
const flagIcon = new FlagIcon()
flagIcon.country = "jp"
document.body.appendChild(flagIcon)
```

Run

- HTML Templates

<template> & **<slot>** tags to write markup templates (✗ advanced as jinja). reusable & structured templates for content

```
<template id="book-template">
  <li><span class="author"></span>'s classic novel
<span class="title"></span></li>
</template>
```

```
<ul id="books"></ul>
```

```
const fragment = document.getElementById('book-
template');
const books = [
  { title: 'The Great Gatsby', author: 'F. Scott
Fitzgerald' },
  { title: 'A Farewell to Arms', author: 'Ernest
Hemingway' },
  { title: 'Catch 22', author: 'Joseph Heller' }
];
```

```
books.forEach(book => {
  // Create an instance of the template content
  const instance =
document.importNode(fragment.content, true);
  // Add relevant content to the template
  instance.querySelector('.title').innerHTML =
book.title;
  instance.querySelector('.author').innerHTML =
book.author;
  // Append the instance to the DOM
```

```
document.getElementById('books').appendChild(instance);
});
```

```
//creates a template (<template id="book-
template">) without any other Web Components
```

technology, three technologies in the stack can be used independently or collectively.

Frameworks

- Python can create network listeners, basic templates, manipulate strings...
Flask, django
- JS can extend elements, APIs to manipulate documents **React.js, Angular.js, vue.js, Ember.js (frontend), Next.js, Node.js**
- SPA: **Single Page Application**  mobile & dynamically update content without reloading.
- Lot of code repetition - **boilerplate** reinventing styles-techniques
- Standard techniques for common problems - **design patterns**

T1 2025

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Home page</title>
  <script>
    //Missing JS code
  </script>
</head>
<body>
  <form>
    Your name : <input type="text" id="txt1" value="Welcome to IITM Online BS
Degree" size="35">
    <br>
    <input type="button" onclick="doSomething()" value="Click me!">
  </form>
</body>
</html>

```

If the user clicks on the “Click me!” button, then which of the following JS codes should be added in the <script> tag that will bold the content of <input> tag as given in the image below?

Your name : **Welcome to IITM Online BS Degree**

OPTIONS :

☐

```
function doSomething(){
  txt1=document.getElementById("txt1");
  txt1.style.fontWeight="bold";
}
```

☐

```
function doSomething(){
  document.getElementById("txt1").style.fontWeight="bold";
}
```

☐

```
function doSomething(){
  txt1=document.getElementById("txt1");
  txt1.style="bold";
}
```

☐

```
function doSomething(){
  document.getElementById("txt1").style="bold";
}
```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function makeBold(){
      //MISSING JS CODE
    }
  </script>
</head>
<body>
  Enter your name : <input type="text" id="txt1" name="txt1">
  <br>
  <input type="button" onclick="makeBold();" value="Click Me">
</body>
</html>

```

Replace the **MISSING JS CODE** part with the correct JS code that will change the text box content to bold?

OPTIONS :

- ☐ `t=document.getElementById("txt1").value;`
`t.style.fontWeight="bold";`
- ☐ `t=document.getElementById("txt1").value;`
`t.style.fontWeight="bold";`
- ☐ `t=document.getElementById("txt1");`
`t.style.fontWeight="bold";`
- ☐ `t=document.getElementById("txt1");`
`t.style.fontWeight="bold";`

T2 2024

Match the following terms with their correct descriptions:

A. DOM Event	1. The structure that represents the HTML document and allows interaction with it.
B. Attributes	2. A method or action triggered by user interactions like clicks, typing, etc.
C. Document	3. The individual elements or parts of a page, such as elements, attributes, or text.
D. Node	4. The properties that define the characteristics of an HTML element, like id, class, etc.
E. DOM (Document Object Model)	5. A programming interface for web documents that provides structured access to the document's elements.

OPTIONS :

- ☐ A → 2, B → 4, C → 1, D → 3, E → 5
- ☐ A → 3, B → 4, C → 1, D → 2, E → 5
- ☐ A → 2, B → 4, C → 5, D → 3, E → 1
- ☐ A → 3, B → 4, C → 5, D → 2, E → 1

T1 2024

Which of the below JavaScript statement(s) are used to print information on the browser's console?

Options :

`window.alert()`

`console.log()`

`document.write()`

None of these